

Flask Website Manual

How drnealaggarwal.info Works

Dr Neal Aggarwal · June 2026

1. Introduction

This manual explains how your personal website at **drnealaggarwal.info** is built and how it works. The site is a pure Python/Flask web application: there is no WordPress, no Wix, no Content Management System. Every page is generated directly by a Python program running on a server, using simple text files as its source material. Understanding this architecture means you understand precisely what happens between a visitor typing your URL and a page appearing in their browser.

The manual is organised as follows: we first explain what Flask is and how it fits into the Python web ecosystem; we then tour every file in your project and explain what it does; we show how a visitor's HTTP request travels through the stack; and finally we explain how Render hosts the site and publishes it to the internet.

2. Python and Flask

2.1 What is a web framework?

When a browser requests a web page, it sends an HTTP request to a server. That server must receive the request, work out which page is wanted, produce the HTML, and send it back. A *web framework* is a Python library that handles all of this infrastructure so you only write the interesting parts: the logic that decides what each URL shows.

2.2 Flask specifically

Flask is a *micro-framework*: it does the minimum necessary and stays out of your way. Its two core concepts are:

- **Routes** — Python functions that are bound to a URL pattern. When a request arrives for that URL, Flask calls the function.
- **Templates** — HTML files with placeholders. Flask fills the placeholders with data from your Python function and sends the result to the browser.

A minimal Flask application looks like this:

```

from flask import Flask, render_template
app = Flask(__name__)

@app.route('/')
def home():
    return render_template('home.html', name='Neal')

if __name__ == '__main__':
    app.run()

```

The `@app.route('/')` line is a Python *decorator*: it registers the function below it as the handler for the root URL. `render_template` loads an HTML file, substitutes `{{ name }}` with 'Neal', and returns the result.

2.3 The request-response cycle

Every page load follows the same sequence:

Step	What happens
1	Visitor types drnealaggarwal.info in their browser
2	Browser asks DNS: 'What is the IP address of drnealaggarwal.info?'
3	DNS replies with Render's IP address
4	Browser sends an HTTP GET request to Render's server
5	Render passes the request to your Python/Flask process
6	Flask matches the URL to a route function
7	The route function reads files (Markdown, JSON) and calls <code>render_template()</code>
8	Flask fills the HTML template with data and returns it
9	Render sends the HTML back to the browser
10	Browser renders the HTML into the visible page

3. Your Website's File Structure

Every file in `drneal-site/` has a specific role. The diagram below shows the complete tree, followed by a file-by-file explanation.

```

drneal-site/
├── app.py           ← Flask application (main program)
├── requirements.txt ← Python package list
├── render.yaml      ← Render deployment config
├── start-site.command ← Double-click to run locally on Mac
├── DEPLOY.md        ← Deployment notes
├── README.md        ← Project overview
├── __pycache__/     ← Python compiled bytecode (auto-generated)
└── content/

```

```
■   ■■■ posts/           ← Blog post Markdown files (.md)
■   ■■■ data/
■       ■■■ library.json ← Book/resource library data
■       ■■■ skills.json  ← Skills and tools data
■       ■■■ demos.json   ← Demo projects data
■■■ static/
    ■■■ Aggarwal_Maasai_Thesis_FINAL.pdf
    ■■■ brinkAndBack.pdf
    ■■■ fastai-lesson3-learning-guide.pdf
    ■■■ fastai-lesson3-learning-guide.epub
    ■■■ audio/           ← Audio files (.m4a)
    ■■■ css/             ← Stylesheet files
    ■■■ img/             ← Images
    ■■■ js/              ← JavaScript files
```

3.1 app.py — The heart of the site

This single Python file *is* the website. At 92 KB it is substantial, but its structure is straightforward. At the top, it imports the libraries it needs:

```
import os, json, re
from pathlib import Path
from datetime import datetime
import smtplib
import markdown
from flask import Flask, render_template, abort, request, redirect, url_for, flash, jsonify
```

Then it creates the Flask application object and sets a secret key (used to sign browser session cookies):

```
app = Flask(__name__)
app.secret_key = os.environ.get('SECRET_KEY', 'dev-secret-neal-site-2026')
```

The `os.environ.get` call means: use the environment variable `SECRET_KEY` if it exists (on Render's server), or fall back to the development default if running locally. This is standard practice — production secrets are never hard-coded into source files.

Routes in `app.py` look like this:

```
@app.route('/post/')
def post(slug):
    # Read content/posts/.md from disk
    # Convert Markdown → HTML using the markdown library
    # Pass the HTML to a Jinja2 template
    return render_template('post.html', content=html, title=title)
```

The angle-bracket syntax `<slug>` in the route is a URL variable: Flask captures whatever appears at that position in the URL and passes it as an argument to the function. So `/post/2026-06-24-maasai-glycaemic-study` calls `post(slug='2026-06-24-maasai-glycaemic-study')`.

The `markdown` import is the Python-Markdown library. It converts the raw text of your blog posts (written in Markdown syntax) into valid HTML that the browser can display. This means you write posts in plain text

and the application handles the HTML formatting automatically.

3.2 requirements.txt — Dependency declaration

This file lists every Python package the application needs. It is read by pip (the Python package installer) at deployment time. A typical requirements.txt for your site looks like:

```
flask
markdown
gunicorn
```

gunicorn is the production web server (see section 5). You never interact with requirements.txt directly — it is used automatically by Render when deploying.

3.3 render.yaml — Deployment configuration

This YAML file tells Render how to build and run your site. It specifies the build command (install dependencies), the start command (launch the Flask app), environment variables, and other settings. A representative render.yaml looks like:

```
services:
- type: web
  name: drneal-site
  runtime: python
  buildCommand: pip install -r requirements.txt
  startCommand: gunicorn app:app
  envVars:
  - key: SECRET_KEY
    generateValue: true
```

The start command `gunicorn app:app` means: use gunicorn to run the object called `app` inside the file `app.py`.

3.4 start-site.command — Local development

This is a macOS shell script you can double-click in Finder to start the site on your own computer. It sets environment variables and runs Flask in development mode, making the site available at `http://localhost:5000`. Nothing in this file affects the live website.

3.5 content/posts/ — Blog posts

Each file in this folder is one blog post, written in Markdown — a plain-text format that uses simple symbols for formatting. Every post file begins with a YAML frontmatter block:

```
---
title: "Insulin Quiescence and Glycaemic Stability in a Maasai Cohort"
date: 2026-06-24
tags: [nutrition, diabetes, metabolism, Maasai]
description: "A field study of 183 Maasai participants..."
---
```

The body of the post goes here, written in Markdown...

When a visitor requests `/post/2026-06-24-maasai-glycaemic-study`, Flask reads this file, strips the frontmatter, converts the Markdown body to HTML, and renders the result into the post template. **To publish a new post, you drop a new .md file into this folder** — no other changes are needed.

3.6 content/data/ — Structured content

These JSON files power the non-blog sections of the site:

File / Folder	Purpose
<code>library.json</code>	Your book and resource library — titles, authors, descriptions, cover images.
<code>skills.json</code>	Tools and skills you work with — names, categories, proficiency levels.
<code>demos.json</code>	Demo projects and tools — names, descriptions, links.

JSON (JavaScript Object Notation) is a structured text format for data. Flask reads these files at request time, parses them into Python dictionaries, and passes the data to the appropriate template. To update your library or skills page, you edit the JSON file and push to GitHub — Render redeploys automatically.

3.7 static/ — Files served directly

The `static/` folder is special in Flask: its contents are served to the browser *as-is*, without any processing. This is where you put files that must be downloaded exactly as stored:

File / Folder	Purpose
<code>static/css/</code>	CSS stylesheets that control the visual appearance of every page.
<code>static/js/</code>	JavaScript files for interactive elements (menus, animations).
<code>static/img/</code>	Images used across the site.
<code>static/audio/</code>	Audio files (.m4a) used in posts.
<code>static/*.pdf</code>	PDF documents (theses, guides, manuals) available for download.

Any file in `static/` is accessible via the URL `/static/filename`. For example, the Maasai thesis is at `https://drnealaggarwal.info/static/Aggarwal_Maasai_Thesis_FINAL.pdf`. **This is why download links in posts must use the `/static/` prefix** — Flask does not serve files from the root URL unless you write a specific route for them.

3.8 __pycache__/ — Auto-generated bytecode

Python compiles your source files to bytecode (.pyc files) the first time they are run, storing them here for faster subsequent loads. You never edit this folder and it is excluded from git (listed in `.gitignore`). If you delete it, Python simply regenerates it on the next run.

3.9 DEPLOY.md and README.md

README.md is a brief project description displayed on GitHub. DEPLOY.md contains your personal notes on deployment steps and configuration that is not captured in render.yaml.

4. Jinja2 Templates — Separating Logic from Presentation

Flask uses the Jinja2 templating engine to separate Python logic from HTML presentation. A template is an HTML file with placeholders written in double braces:

```
<!-- templates/post.html -->
<h1>{{ title }}</h1>
<p class="date">{{ date }}</p>
<div class="content">
  {{ content | safe }}
</div>
```

The `{{ title }}` syntax inserts the value of the `title` variable passed from your route function. The `| safe` filter tells Jinja2 not to escape HTML characters — necessary because content is already rendered HTML from the Markdown converter.

Jinja2 also supports control flow:

```
{% for post in posts %}
  <article>
    <h2><a href="/post/{{ post.slug }}">{{ post.title }}</a></h2>
    <p>{{ post.description }}</p>
  </article>
{% endfor %}
```

This generates one `<article>` block for every post in the list — the template loops through the data provided by Flask and produces the corresponding HTML. Templates are stored in a `templates/` folder inside the project root.

5. How the Site Appears on the Internet

5.1 DNS — The internet's address book

Every device on the internet has an IP address: a numeric identifier like 216.24.57.3. Domain names like `drnealaggarwal.info` are human-readable aliases for those numbers. DNS (Domain Name System) is the distributed database that maps names to numbers.

When a visitor types your URL, their browser queries a DNS resolver (typically provided by their internet service provider). The resolver follows a chain of DNS servers until it reaches the authoritative nameserver for `.info` domains, which returns the IP address configured in your domain registrar's DNS settings. This address points to Render's infrastructure.

5.2 HTTP/HTTPS — The conversation protocol

HTTP (HyperText Transfer Protocol) is the language browsers and servers use to communicate. When a browser has the IP address, it opens a TCP connection to port 443 (HTTPS) and sends a request like:

```
GET /post/2026-06-24-maasai-glycaemic-study HTTP/1.1
Host: drnealaggarwal.info
Accept: text/html
```

The S in HTTPS means the connection is encrypted using TLS (Transport Layer Security). Render handles TLS automatically, provisioning and renewing SSL certificates via Let's Encrypt. Your Python code never sees unencrypted traffic.

5.3 Gunicorn — The production web server

Flask's built-in development server is single-threaded and not designed for production traffic. On Render, your application runs under **Gunicorn** (Green Unicorn), a production-grade Python WSGI server. Gunicorn:

- Spawns multiple *worker processes* so concurrent requests are handled simultaneously.
- Acts as the bridge between Render's HTTP infrastructure and your Flask app.
- Follows the WSGI specification — the standard interface between Python web apps and web servers.

The chain is: **Browser** → **Render load balancer** → **Gunicorn** → **Flask** → **your code**.

6. How Render Works

6.1 What Render is

Render is a cloud platform-as-a-service (PaaS). It sits between raw cloud infrastructure (like AWS virtual machines) and the complexity of configuring servers manually. You give Render a GitHub repository; Render builds, deploys, and serves your application on managed infrastructure. You pay for compute time and never touch a server terminal.

6.2 The deployment pipeline

Every time you push a commit to your GitHub repository's master branch, the following sequence runs automatically:

1. Trigger

Render's GitHub webhook fires within seconds of your git push.

2. Clone

Render checks out the latest commit of your repository on a fresh build server.

3. Build

Render runs your build command: `pip install -r requirements.txt`. This installs Flask, markdown, gunicorn, and any other dependencies.

4. Health check

Render starts the new instance and waits for it to respond to HTTP requests successfully.

5. Swap

Once the new instance is healthy, Render routes all incoming traffic to it and terminates the old instance. This gives zero-downtime deploys.

6.3 Environment variables on Render

Secrets like `SECRET_KEY` and `GMAIL_APP_PASS` (for the contact form) are stored in Render's dashboard as environment variables, not in your code. When the application starts, Render injects them into the process environment. Your Python code reads them with `os.environ.get('SECRET_KEY')`. This means the secret values never appear in your GitHub repository.

6.4 Persistent vs. ephemeral storage

Render's file system is *ephemeral*: anything written to disk during a deployment is lost when the next deployment happens. This means you cannot use the server's local disk to store user uploads or any data you want to keep. For your site this is not an issue — all content lives in your GitHub repository (Markdown files, JSON files, static assets) and is deployed fresh with each push.

6.5 Custom domain

Render assigns your service a subdomain like `drneal-site.onrender.com`. To use `drnealaggarwal.info`, you added a CNAME or A record in your domain registrar's DNS settings pointing to Render's servers. Render detects the custom domain and automatically provisions an SSL certificate for it.

7. Worked Example: Loading a Blog Post

This section traces a single request through every layer of the stack.

Visitor	action:	The	visitor	types
<code>https://drnealaggarwal.info/post/2026-06-24-maasai-glycaemic-study</code>			into	their
browser.				

Step 1 — DNS resolution

The browser asks a DNS resolver for the IP of `drnealaggarwal.info`. The resolver returns Render's IP address.

Step 2 — TLS handshake

The browser opens a connection to port 443. Render's TLS termination layer presents the SSL certificate for `drnealaggarwal.info`. An encrypted channel is established.

Step 3 — HTTP request to Gunicorn

The browser sends the HTTP GET request. Render forwards it to the Gunicorn worker assigned to your service.

Step 4 — Flask routing

Gunicorn passes the request to Flask. Flask matches the path `/post/2026-06-24-maasai-glycaemic-study` against its registered routes and calls the `post(slug='2026-06-24-maasai-glycaemic-study')` function.

Step 5 — File reading and Markdown conversion

The route function constructs the file path `content/posts/2026-06-24-maasai-glycaemic-study.md`, reads it from disk, extracts the YAML frontmatter (title, date, tags, description), and passes the body text to the Python-Markdown library. Markdown converts it to HTML.

Step 6 — Template rendering

Flask calls `render_template('post.html', content=html, ...)`. Jinja2 loads the template, substitutes all variables, and returns a complete HTML string.

Step 7 — HTTP response

Flask wraps the HTML in an HTTP 200 response and returns it through Gunicorn → Render → browser. The browser parses the HTML, requests the referenced CSS and JavaScript files (served from `/static/`), and renders the page.

Step 8 — Static asset requests

The browser notices `<link href='/static/css/main.css'>` and sends a second request for that file. Flask's built-in static file handler reads `static/css/main.css` from disk and returns it directly — no template rendering required.

8. Day-to-Day Site Maintenance

8.1 Publishing a new blog post

1. Create a new `.md` file in `drneal-site/content/posts/` with the naming convention `YYYY-MM-DD-slug.md`.
2. Begin the file with YAML frontmatter (title, date, tags, description).
3. Write the post body in Markdown.
4. Stage and commit the file in GitHub Desktop.
5. Push to master. Render redeploys automatically within 1–2 minutes.

8.2 Adding a PDF or other static asset

1. Copy the file into `drneal-site/static/`.
2. Link to it in your post or page using the `/static/filename.pdf` URL — never a bare root URL.
3. Commit and push.

8.3 Updating library or skills data

Edit the relevant JSON file in `drneal-site/content/data/`. JSON must be valid (no trailing commas, strings in double quotes). Commit and push.

8.4 Running the site locally

Double-click `start-site.command` in Finder. The site starts at `http://localhost:5000`. Changes to Python files and templates take effect immediately in development mode without restarting.

8.5 Checking deployment status

Log in to `dashboard.render.com`. Your service shows a live deploy log. If a deploy fails (e.g. a Python syntax error), Render retains the previous working version and marks the failed deploy in red. The error message in the log pinpoints the problem.

9. Glossary

Term	Definition
DNS	Domain Name System. Maps human-readable domain names to IP addresses.
Flask	A Python micro web framework. Handles HTTP routing, template rendering, and static file serving.
Gunicorn	Green Unicorn. A production Python WSGI server that runs your Flask app with multiple worker processes.
HTTP/HTTPS	HyperText Transfer Protocol. The request/response protocol browsers use to fetch pages. HTTPS adds TLS encryption.
Jinja2	Flask's default HTML templating engine. Uses <code>{{ }}</code> for variable substitution and <code>{% %}</code> for control flow.
JSON	JavaScript Object Notation. A plain-text data format for structured data (lists and key-value pairs).
Markdown	A lightweight markup language. Plain text with simple formatting conventions that converts to HTML.
PaaS	Platform-as-a-Service. Cloud hosting where the provider manages the server infrastructure.
Render	The cloud platform that hosts your site. Connects to GitHub, builds on push, and serves via HTTPS.
Route	A Flask function registered to handle requests for a specific URL pattern.
SSL/TLS	Encryption protocol for HTTPS connections. Render provisions certificates automatically.
Static files	Files served to the browser unchanged. CSS, JS, images, PDFs. Stored in <code>static/</code> and served at <code>/static/</code> .
Template	An HTML file with Jinja2 placeholders. Flask fills the placeholders with data at request time.

WSGI	Web Server Gateway Interface. The Python standard defining how web servers communicate with Python apps.
YAML	A human-readable data format used for configuration files and Markdown frontmatter.

This manual was generated automatically from the live drneal-site repository and accurately reflects the file structure and architecture as of 24 June 2026. It is stored at /static/flask_website_manual.pdf on the server and is not linked from any page.